Lab Assignment-11.3

Name: Goutham Reddy

Hall Ticket no: 2303A51345

Batch No: 20

Task -1:

Prompt:

Task 1: Smart Contact Manager (Arrays & Linked Lists)
Scenario
SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.
Tasks
1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
o Add a contact
o Search for a contact
o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
o Insertion efficiency
o Deletion efficiency

```python
11.3-Assignment.py > ...
from collections import deque
import heapq

"""
Contact Manager Application
A simple contact management system using a list to store contacts.
"""
class ContactManager:
    """A class to manage contacts with add, search, delete, and display operations."""

    def __init__(self):
        """Initialize an empty contacts list."""
        self.contacts = []

    def add_contact(self, name, phone):
        """
        Add a new contact to the manager.

        Args:
            name (str): The contact's name
            phone (str): The contact's phone number
        """
        contact = {"name": name, "phone": phone}
        self.contacts.append(contact)
        print(f"✓ Contact '{name}' added successfully!\n")

    def search_contact(self, name):
        """
        Search for a contact by name.

        Args:
            name (str): The name of the contact to search for

        Returns:
            dict: The contact if found, None otherwise
        """
        for contact in self.contacts:
            if contact["name"].lower() == name.lower():
                return contact
        return None

    def delete_contact(self, name):
        """
        Delete a contact by name
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Programs\Python\Python314\pyt
gnment.py
--- Adding Contacts ---

✓ Contact 'Alice Johnson' added successfully!

✓ Contact 'Bob Smith' added successfully!

✓ Contact 'Carol White' added successfully!

--- All Contacts ---

========================================
Name                 | Phone
========================================
Alice Johnson        | 555-0101
Bob Smith            | 555-0102
Carol White          | 555-0103
========================================

--- Searching for 'Bob Smith' ---
Found: Bob Smith - 555-0102

--- Deleting 'Carol White' ---
✓ Contact 'Carol White' deleted successfully!

--- Updated Contacts ---

========================================
Name                 | Phone
========================================
Alice Johnson        | 555-0101
Bob Smith            | 555-0102
========================================

PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

The array implementation is simple and allows quick access using indexing, but deletion requires shifting elements. The linked list supports dynamic memory allocation and efficient insertions or deletions without shifting. However, searching requires traversal. This comparison highlights performance differences in insertion and deletion efficiency.

Task 2:

Prompt:

Library Book Search System (Queues & Priority Queues)
Scenario
The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.
Tasks
1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:
o enqueue() method
o dequeue() method
4. Test the system with a mix of student and faculty requests.

```python
#Task-2

class Node:
    """A node in the linked list representing a single contact."""

    def __init__(self, name, phone):
        """
        Initialize a node with contact information.

        Args:
            name (str): The contact's name
            phone (str): The contact's phone number
        """
        self.name = name
        self.phone = phone
        self.next = None


class LinkedListContactManager:
    """A contact manager using a linked list data structure."""

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def add_contact(self, name, phone):
        """
        Add a new contact to the end of the linked list.

        Args:
            name (str): The contact's name
            phone (str): The contact's phone number
        """
        new_node = Node(name, phone)

        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Programs\Pyt
gnment.py
--- Adding Contacts ---

✓ Contact 'Alice Johnson' added successfully!

✓ Contact 'Bob Smith' added successfully!

✓ Contact 'Carol White' added successfully!

--- All Contacts ---

========================================
Name                 | Phone
========================================
Alice Johnson        | 555-0101
Bob Smith            | 555-0102
Carol White          | 555-0103
========================================

--- Searching for 'Bob Smith' ---
Found: Bob Smith - 555-0102

--- Deleting 'Carol White' ---
✓ Contact 'Carol White' deleted successfully!

--- Updated Contacts ---

========================================
Name                 | Phone
========================================
Alice Johnson        | 555-0101
Bob Smith            | 555-0102
========================================

--- Adding Mixed Faculty and Student Requests ---

✓ Student request from 'John Smith' added (ID: 1)

✓ Faculty request from 'Dr. Alice Brown' added (ID: 2)

✓ Student request from 'Maria Garcia' added (ID: 3)
```

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Progr
gnment.py
✓ Faculty request from 'Dr. Sarah Davis' added (ID: 6)

--- Current Request Queue (Priority Order) ---

=====================================================
Position   | Role       | Name            | Priority
=====================================================
1          | Faculty    | Dr. Alice Brown | Faculty
2          | Faculty    | Prof. James Wilson | Faculty
3          | Faculty    | Dr. Sarah Davis | Faculty
4          | Student    | John Smith      | Student
5          | Student    | Maria Garcia    | Student
6          | Student    | Robert Lee      | Student
=====================================================

--- Processing Requests (Faculty First) ---

✓ Processing: Faculty request from 'Dr. Alice Brown'

✓ Processing: Faculty request from 'Prof. James Wilson'

✓ Processing: Faculty request from 'Dr. Sarah Davis'

--- Remaining Requests ---

=====================================================
Position   | Role       | Name            | Priority
=====================================================
1          | Student    | John Smith      | Student
2          | Student    | Maria Garcia    | Student
3          | Student    | Robert Lee      | Student
=====================================================

--- Processing Remaining Student Requests ---

✓ Processing: Student request from 'John Smith'

✓ Processing: Student request from 'Maria Garcia'

✓ Processing: Student request from 'Robert Lee'
```

Justification:

The Queue ensures requests are processed in FIFO order, maintaining fairness. The Priority Queue improves the system by prioritizing faculty requests over student requests. This demonstrates how selecting appropriate data structures affects system efficiency and real-world applicability in managing institutional workflows.

Task-3:

Prompt:

Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.
2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

11.3-Assignment.py > ...

```python
#Task-3

class Ticket:
    """Represents a single IT help desk ticket."""

    def __init__(self, ticket_id, issue, priority):
        """
        Initialize a ticket.

        Args:
            ticket_id (str): Unique ticket identifier
            issue (str): Description of the issue
            priority (str): Priority level (Low, Medium, High)
        """
        self.ticket_id = ticket_id
        self.issue = issue
        self.priority = priority

    def __repr__(self):
        return f"Ticket({self.ticket_id}, {self.issue}, {self.priority})"


class HelpDeskStack:
    """A stack-based help desk ticket management system using LIFO principle."""

    def __init__(self, max_size=10):
        """
        Initialize an empty stack with maximum size.

        Args:
            max_size (int): Maximum number of tickets the stack can hold
        """
        self.tickets = []
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Programs\Python\Python31
gnment.py
--- Pushing 5 Tickets (LIFO Order) ---

✓ Ticket T001 added to stack

✓ Ticket T002 added to stack

✓ Ticket T003 added to stack

✓ Ticket T004 added to stack

✓ Ticket T005 added to stack

--- Current Stack ---

==================================================
Position   | Ticket ID    | Issue           | Priority
==================================================
5          | T005         | Hardware Issue  | High
4          | T004         | Email Config    | Medium
3          | T003         | Printer Setup   | Medium
2          | T002         | Network Issue   | High
1          | T001         | Password Reset  | Low
==================================================

--- Peek at Top Ticket ---
Top ticket: Ticket(T005, Hardware Issue, High)

--- Processing Tickets (LIFO - Last In, First Out) ---

✓ Ticket T005 processed and removed

✓ Ticket T004 processed and removed

--- Stack After 2 Pops ---

==================================================
Position   | Ticket ID    | Issue           | Priority
==================================================
3          | T003         | Printer Setup   | Medium
2          | T002         | Network Issue   | High
```

```
✓ Ticket T005 processed and removed

✓ Ticket T004 processed and removed

--- Stack After 2 Pops ---

==================================================
Position   | Ticket ID    | Issue           | Priority
==================================================
3          | T003         | Printer Setup   | Medium
2          | T002         | Network Issue   | High
1          | T001         | Password Reset  | Low
==================================================

✓ Ticket T003 processed and removed

✓ Ticket T002 processed and removed

✓ Ticket T001 processed and removed

--- Try to Pop from Empty Stack ---
✗ Stack is empty! No tickets to process

PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

The Stack follows the LIFO principle, making it suitable for ticket escalation systems. The most recent ticket is resolved first, ensuring urgent issues receive attention. Stack operations like push, pop, and peek clearly demonstrate LIFO behavior and structured issue management.

Task 4:

Prompt: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

- Insert

• Search
• Delete
Starter Code
class HashTable:
pass

```python
#Task-4

class HashTable:
    """A hash table implementation using chaining to handle collisions."""

    def __init__(self, size=10):
        """
        Initialize a hash table with a given size.

        Args:
            size (int): Number of buckets in the hash table
        """
        self.size = size
        # Each bucket is a list that stores key-value pairs
        # This list handles collisions through chaining
        self.buckets = [[] for _ in range(size)]

    def _hash(self, key):
        """
        Simple hash function using modulo operation.

        Args:
            key (str): The key to hash

        Returns:
            int: Bucket index (0 to size-1)
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.
        Handles collisions by chaining (storing multiple pairs in same bucket).

        Args:
            key (str): The key to insert
            value: The value associated with the key
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Programs\Python\
gnment.py
--- Inserting Data ---

✓ Inserted key 'name' with value 'Alice'
✓ Inserted key 'email' with value 'alice@example.com'
✓ Inserted key 'phone' with value '555-0101'
✓ Inserted key 'age' with value '30'
✓ Inserted key 'city' with value 'New York'

--- Hash Table Contents ---

==================================================
Bucket     | Key            | Value
==================================================
0          | email          | alice@example.com
1          | phone          | 555-0101
2          | name           | Alice
3          | (empty)        |
4          | age            | 30
4          | city           | New York
==================================================

--- Searching for Values ---

Search 'email': alice@example.com

Search 'nonexistent': None

--- Deleting a Key ---

✓ Deleted key 'age'

--- Updated Hash Table ---

==================================================
Bucket     | Key            | Value
==================================================
0          | email          | alice@example.com
1          | phone          | 555-0101
2          | name           | Alice
3          | (empty)        |
4          | city           | New York
==================================================
```

Justification:

The Hash Table implementation uses chaining to handle collisions efficiently. Insert, search, and delete operations are optimized for fast access. Chaining prevents data loss during collisions and

improves performance. This demonstrates practical understanding of hashing techniques and collision resolution strategies.

Task 5:

Prompt:

Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

11.3-Assignment.py > ...

```python
# Task-5: Data Structure Recommendations
recommendations = """
Feature | Data Structure | Justification
--------|----------------|--------------
Event Registration System | Queue (FIFO) | Processes registrations in order of arrival, e
Library Book Borrowing | Linked List | Allows efficient insertion/deletion of borrowed bo
Bus Scheduling System | Stack (LIFO) | Manages bus routes in reverse chronological order,
Cafeteria Order Queue | Queue (FIFO) | Processes food orders sequentially in the order re
"""

print(recommendations)

class Order:
    """Represents a single cafeteria order."""

    def __init__(self, order_id, customer_name, items):
        """
        Initialize an order.

        Args:
            order_id (str): Unique order identifier
            customer_name (str): Name of the customer
            items (list): List of food items ordered
        """
        self.order_id = order_id
        self.customer_name = customer_name
        self.items = items

    def __repr__(self):
        return f"Order({self.order_id}, {self.customer_name}, {self.items})"


class CafeteriaOrderQueue:
    """A queue-based cafeteria order management system using FIFO principle."""

    def __init__(self):
        """Initialize an empty queue using deque for efficient O(1) operations."""
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Programs\Python\Python314\python.exe c:/Users/temba/OneDr
gnment.py

Feature | Data Structure | Justification
--------|----------------|--------------
Event Registration System | Queue (FIFO) | Processes registrations in order of arrival, ensuring fair registration and maintaining the seque
Library Book Borrowing | Linked List | Allows efficient insertion/deletion of borrowed books and maintains a dynamic list of transactions wi
Bus Scheduling System | Stack (LIFO) | Manages bus routes in reverse chronological order, useful for cancellations and tracking the most rec
Cafeteria Order Queue | Queue (FIFO) | Processes food orders sequentially in the order received, ensuring customers are served fairly and ma

--- Placing 5 Orders (FIFO Order) ---

✓ Order ORD001 placed for Alice: Burger, Fries, Coke

✓ Order ORD002 placed for Bob: Pizza, Salad

✓ Order ORD003 placed for Carol: Sandwich, Ice Cream

✓ Order ORD004 placed for David: Pasta, Garlic Bread, Juice

✓ Order ORD005 placed for Eve: Tacos, Beans

--- Current Order Queue ---

============================================================
Position   | Order ID   | Customer        | Items
============================================================
1          | ORD001     | Alice           | Burger, Fries, Coke
2          | ORD002     | Bob             | Pizza, Salad
3          | ORD003     | Carol           | Sandwich, Ice Cream
4          | ORD004     | David           | Pasta, Garlic Bread, Juice
5          | ORD005     | Eve             | Tacos, Beans
============================================================

--- Serving Orders (FIFO - First In, First Out) ---

✓ Serving Order ORD001 for Alice: Burger, Fries, Coke

✓ Serving Order ORD002 for Bob: Pizza, Salad

--- Queue After Serving 2 Orders ---
```

```
--- Serving Orders (FIFO - First In, First Out) ---

✓ Serving Order ORD001 for Alice: Burger, Fries, Coke

✓ Serving Order ORD002 for Bob: Pizza, Salad

--- Queue After Serving 2 Orders ---

============================================================
Position   | Order ID   | Customer        | Items
============================================================
1          | ORD003     | Carol           | Sandwich, Ice Cream
2          | ORD004     | David           | Pasta, Garlic Bread, Juice
3          | ORD005     | Eve             | Tacos, Beans
============================================================

--- Serving Remaining Orders ---

✓ Serving Order ORD003 for Carol: Sandwich, Ice Cream

✓ Serving Order ORD004 for David: Pasta, Garlic Bread, Juice

✓ Serving Order ORD005 for Eve: Tacos, Beans

--- Try to Serve from Empty Queue ---
✗ No orders to serve. Queue is empty!

PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

Different features require different data structures for efficiency. For example, attendance tracking uses arrays, event registration uses lists, library borrowing uses queues, bus scheduling uses linked lists, and cafeteria orders use queues. Choosing appropriate structures improves system organization, performance, and scalability.